R1-A2-S5 Prepara tu nave: Desde las plataformas empresariales hasta las nubes. Entrega evidencia preparación plataformas proyecto.

Juan Diego Quijano Gutiérrez, Kevin Nicolas López Fonseca

Facultad de Ingeniería, Universidad de Cundinamarca

Ingeniería de Sistemas y Computación

Ing. German Ortiz

18 de marzo de 2026

Contenido

1. Introducción 3

2. Objetivo 4

2.1 OBTEJITIVO GENERAL 4

2.2 OBJETIVOS ESPECIFICOS 4

3. Problema a resolver 4

4. Metodología y Scrum 5

5. Historias de usuario 5

6. Product backlog 6

7. Modelo Entidad-Relación 7

8. UML 7

8.1 Diagrama casos de uso 7

Diagrama de clases 8

9. Evidencia 9

9.1 Evidencia Azure 9

9.2 Evidencia Github 9

9.3 Evidencia modulo del sistema 10

10. Enlaces 10

1. Introducción

La calidad de código es un factor fundamental en el desarrollo de software ya que influye en la mantenibilidad, la eficiencia y la confiabilidad del software sin embargo muchos estudiantes en formación de desarrollo de software presentan una dificultad para identificar errores estructurales y malas practicas en sus programas lo que les podría afectar el rendimiento y la escalabilidad de sus proyectos.

En este contexto, surge la necesidad de contar con una herramienta que permita evaluar su código y le brinde una retroalimentación por esto proponemos el desarrollo de Code Analyzer un software enfocado en el análisis de código en forma estática el cual integra inteligencia artificial.

El sistema va permitir seleccionar archivos o carpetas que contengan código Java analizara su estructura interna y evaluaría métricas como sus líneas de código y su complejidad ciclomatica esta se basara en estándares de ISO 25000 con este análisis el sistema va generar un puntaje de calidad clasificando el código y dándole recomendaciones orientadas a la mejora de las buenas practicas en la programación.

Además, el software va contar con una interfaz amigable que facilite su uso a los estudiantes sin necesidad de tener conocimiento avanzado y que permita generar reportes JSON para su posterior uso y estabilidad, con esto el proyecto busca mejorar el aprendizaje en programación y promover el desarrollo de software de calidad.

2. Objetivo

2.1 OBTEJITIVO GENERAL

Desarrollar una aplicación para proyectos de java que analice el código mediante técnicas de análisis estático y inteligencia artificial con el fin de identificar problemas de calidad en el código y generar recomendaciones que ayuden a mejorar las buenas prácticas de programación de los estudiantes.

2.2 OBJETIVOS ESPECIFICOS

Identificar las necesidades que existen entre los estudiantes relacionado con el código en sus proyectos proponiendo una herramienta que apoye el aprendizaje de buenas prácticas.

Diseñar la arquitectura del sistema y la estructura definiendo que va evaluar del código además planificar la interfaz gráfica y la integración de inteligencia artificial.

Desarrollar la aplicación de análisis de código (Code Analyzer) creando una interfaz gráfica sencilla que permita al usuario visualizar los resultados del análisis.

Probar el funcionamiento del sistema usando proyectos de prueba verificando el análisis del código y las recomendaciones funcionen correctamente.

3. Problema a resolver

En el desarrollo de software muchos estudiantes presentan problemas para identificar errores y malas prácticas en su código lo que puede afectar la calidad, mutabilidad y eficiencia en sus softwares además la revisión de sus docentes puedes llegar a ser limitado, reduciendo la retroalimentación oportuna.

Por otro lado, las herramientas profesionales de análisis suelen ser complejas y poco accesibles Por ello nace la necesidad de desarrollar una solución que permita analizar

automáticamente el código java evalúe su calidad de recomendaciones que apoye su aprendizaje y mejore sus prácticas en programación.

4. Metodología y Scrum

Para el desarrollo del proyecto se utilizaran la combinación de dos metodología de prototipado y el marco de trabajo ágil Scrum con el fin de desarrollar un proyecto iterativo y flexible.

Por el lado de la metodología de prototipado tenemos que nos permite construir versiones que nos facilita la validación de requisitos y el diseño de interfaz a través de estos prototipos se puedes identificar mejoras, corrección de errores y ajustar funcionalidades antes de la entrega del producto final asegurándose de satisfacer las necesidades de los usuarios.

Por el otro lado con Scrum como marco de trabajo ágil podemos organizar iteraciones de desarrollo (Sprints) en este enfoque definimos historias de usuario que representan la funcionalidad del sistema las cuales se organizan en un backlog priorizando durante cada sprint el equipo desarrolla unas funcionalidades permitiendo entregar avances progresivos.

La integración de estas dos nos permite que el proyecto avance de forma incremental combinando la validación continua del prototipo y el seguimiento estructurado propuesto por Scrum.

5. Historias de usuario

Tabla 1

Historias de Usuario

ID	Historia de Usuario	Prioridad
HU-01	Como estudiante de ingeniería de sistemas, quiero que me analice el código de Java automáticamente para ver si tiene problemas mi código.	Alta

ID	Historia de Usuario	Prioridad
HU-02	Como usuario quiero poder recibir recomendaciones de forma automática para tener una retroalimentación de mi código.	Alta
HU-03	Como desarrollador quiero tener repositorios para poder tener un control de versiones.	Alta
HU-04	Como desarrollador, quiero que el sistema exporte los resultados para que realizar un análisis.	Media
HU-05	Como usuario, quiero que el sistema me muestre visualmente los resultados para realizar una autoevaluación de mi código.	Baja

6. Product backlog

Tabla 3

Product backlog

ID	Historia de Usuario	Prioridad	Duración (Semanas)
HU-01	Como estudiante de ingeniería de sistemas, quiero que me analice el código de Java automáticamente para ver si tiene problemas mi código.	Alta	3
HU-02	Como usuario quiero poder recibir recomendaciones de forma automática para tener una retroalimentación de mi código.	Alta	3
HU-03	Como desarrollador quiero tener repositorios para poder tener un control de versiones.	Alta	3
HU-04	Como desarrollador, quiero que el sistema exporte los resultados para que realizar un análisis.	Media	3
HU-05	Como usuario, quiero que el sistema me muestre visualmente los resultados para realizar una autoevaluación de mi código.	Baja	2

7. Modelo Entidad-Relación

No usamos base de datos porque el sistema no requiere una comprensión de datos compleja y el análisis es inmediato y los resultados se almacenan en JSON, lo cual es suficiente y más eficiente para este tipo de herramienta.

8. UML

8.1 Diagrama casos de uso

Figura 1

Diagrama Casos De Uso

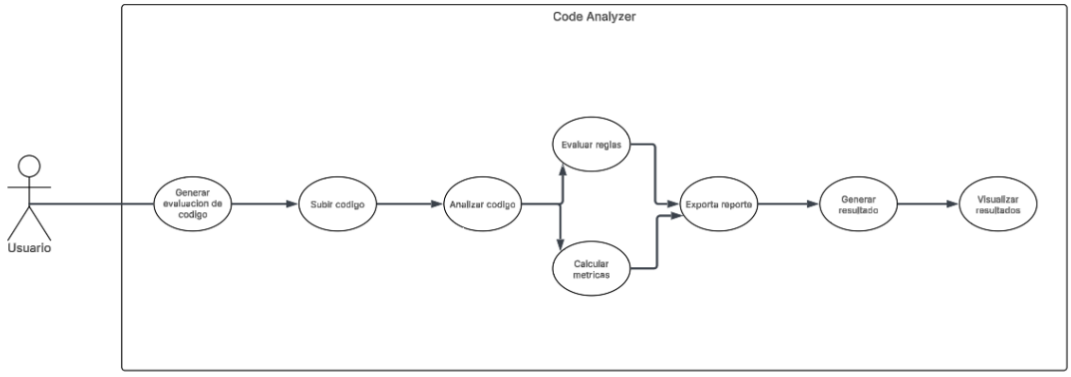
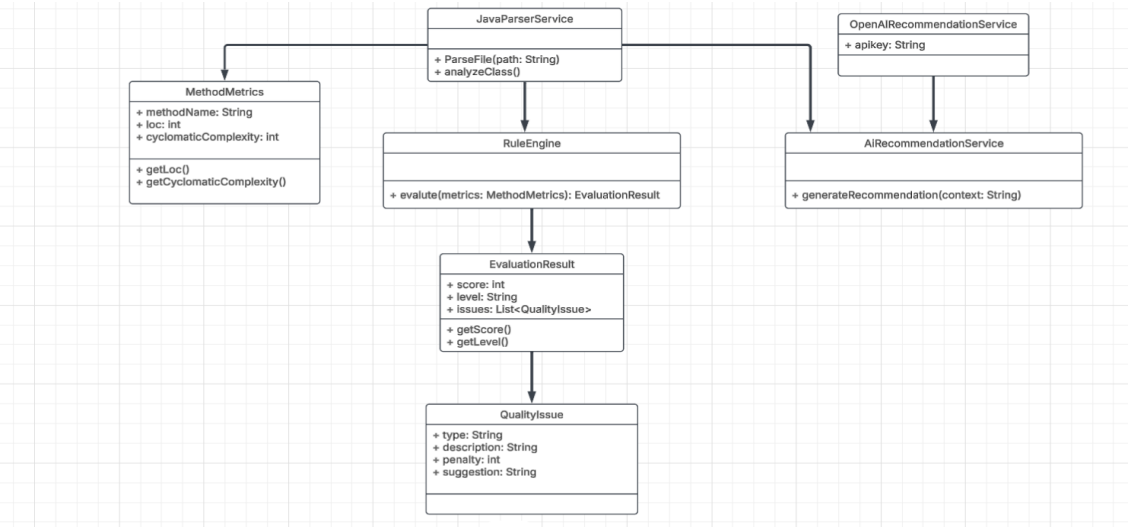


Diagrama de clases

Figura 2

Diagrama De Clases

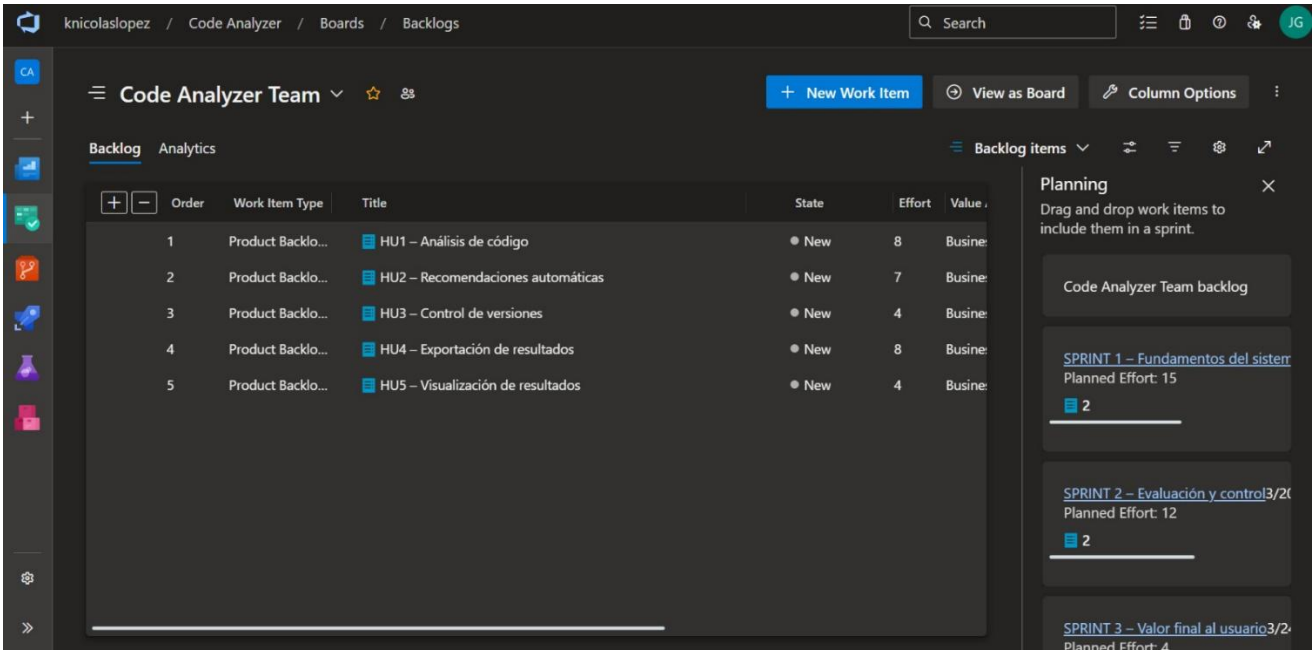


9. Evidencia

9.1 Evidencia Azure

Figura 4

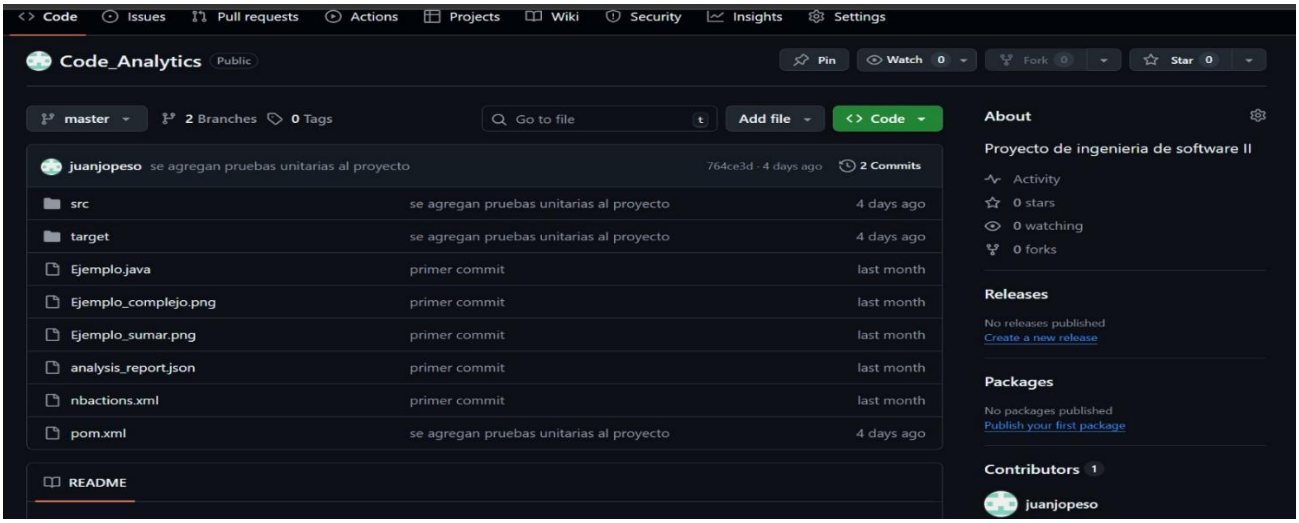
Pantallazo Azure



9.2 Evidencia Github

Figura 5

Pantallazo Github



9.3 Evidencia modulo del sistema

Figura 6

Pantallazo Modulo

```
------------------------
 Método: main
   LOC: 18 | CC: 3
   Puntaje: 100
   Nivel: null
   CFG:
      START -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> STMT
      STMT -> IF
      IF -> THEN
      IF -> ELSE
      JOIN -> STMT
      STMT -> END
  Sin problemas detectados
```

10. Enlaces

Enlace Azure: https://dev.azure.com/knicolaslopez/Code%20Analyzer

Enlace Github: https://github.com/juanjopeso/Code_Analytics